

Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias

Analizador Sintáctico

Triton GPU Kernel

TC3002B — Computer Science Advanced Applications Development
Módulo #3: Compiladores — Fase de Análisis Sintáctico

Equipo: 11

Cristóbal Camarena Hernández — A01642653

Josué Galindo Gutiérrez — A01637983

Gandhi Emmanuel Valdez Huerta — A01625738

Diego Alberto Estrada López — A01638657

Profesor: Dr. Salvador Hinojosa

Fecha de entrega: Junio 2026

Guadalajara, Jalisco, México

Resumen

Resultados: el analizador sintáctico `triton_parser` obtiene **11/11 PASS** en la suite unitaria y **68/70 PARSE_OK (97.1 %)** en el benchmark TRITONHEAL. Sobre kernels válidos, la tasa de aceptación es **100 % (68/68)**; los 2 rechazos son mutaciones inválidas (`k_0040`, `k_0058`) confirmadas con `ast.parse`. Este reporte documenta la CFG, el diseño yacc/Flex, las tablas de símbolos, los mensajes de error y la evidencia experimental con gráfica y tablas de resultados.

Índice

1. Introducción	1
1.1. Resumen	1
1.2. Notación	1
1.3. Justificación de herramientas	1
1.4. Trazabilidad a la fase léxica	1
2. Análisis	1
2.1. Entregables del reto (Sección IV, documento de evaluación)	1
2.2. Requerimientos funcionales	2
2.3. CFG (resumen)	2
2.4. Pasos para obtener la gramática	2
2.5. Simplificaciones justificadas	3
2.6. Modificaciones al analizador léxico	3
2.7. Trazabilidad token léxico $\rightarrow$ yacc	3
3. Diseño	3
3.1. Arquitectura del sistema	3
3.2. Integración lex $\leftrightarrow$ yacc	3
3.3. Diseño de la especificación yacc	4
3.4. Tabla de símbolos del parser (diseño)	4
3.5. Mensajes de error (diseño)	4
3.6. Trazabilidad diseño $\rightarrow$ análisis	4
4. Implementación	4
4.1. Estructura del proyecto	4
4.2. Compilación y ejecución	5
4.3. Fragmento: integración de indentación (lexer)	5
4.4. Fragmento: reglas principales (yacc)	6
4.5. Fragmento: tabla de símbolos	7
4.6. Fragmento: manejo de errores	8
4.7. Ejemplo de salida del parser (entregable 5)	9
5. Verificación y Validación	9
5.1. Resultados experimentales	9
5.2. Resumen cuantitativo	9
5.3. Interpretación de los resultados	9
5.4. Gráfica de resultados (Figura 2)	10
5.5. Detalle de los 2 rechazos (comportamiento esperado)	12
5.6. Modelo de pruebas	12
5.7. Suite unitaria (<code>make test</code>)	12
5.8. Evidencia: salida exitosa (<code>ejemplo_1.tri</code>)	12
5.9. Evidencia: error sintáctico (<code>def_sin_dos_puntos.tri</code>)	13
5.10. Reproducibilidad	13

6. Plan de Trabajo	13
6.1. Distribución de responsabilidades	13
A. Gramática libre de contexto completa	13
A.1. Terminales del analizador	14
A.2. Precedencia y asociatividad (yacc)	14
A.3. Producciones (33 no terminales)	14
A.4. Trazabilidad CFG → implementación	16
B. Referencias	17

Índice de figuras

1. Pipeline del analizador sintáctico.	3
2. Resultados del analizador sintáctico. Panel superior: indicadores clave. Centro izquierda: desglose (68 válidos aceptados, 2 mutaciones rechazadas, 0 falsos rechazos). Centro derecha: 97.1 % PARSE_OK global. Inferior: mapa de los 70 kernels (k_0000–k_0069); verde = éxito, naranja = mutación inválida (k_0040, k_0058).	11

Índice de cuadros

1. Simplificaciones de la CFG.	3
2. Correspondencia Token ID (reporte léxico) y yacc.	3
3. Esquema de entradas en <code>syntab.c</code>	4
4. Cuadro resumen de resultados (corridas locales reproducibles con <code>make test</code> y <code>make benchmark</code>).	9
5. Kernels rechazados: mutaciones inválidas del benchmark.	12
6. Casos de prueba del equipo (11 ejecuciones).	12
7. Cronograma de la fase sintáctica (Equipo 11).	13
8. Conjunto de terminales (yacc / reporte léxico).	14
9. Orden de precedencia (<code>%left</code> / <code>%nonassoc</code>).	14

1. Introducción

1.1. Resumen

Este documento presenta el análisis, diseño, implementación y verificación del **analizador sintáctico** para kernels **Triton GPU** escritos en archivos `.tri`. La fase sintáctica es la segunda etapa del compilador del reto académico: consume la secuencia de tokens producida por el analizador léxico (Flex) del Equipo 11, construye una estructura conforme a una gramática libre de contexto (CFG) implementada con **yacc/Bison**, actualiza tablas de símbolos y reporta errores sintácticos.

El entregable incluye: (i) gramática documentada (Apéndice A, CFG completa); (ii) tablas de símbolos; (iii) implementación `triton_parser`; (iv) mensajes de error; (v) **resultados experimentales** con énfasis en la Sección 5.1: **100 %** suite unitaria, **97.1 %** benchmark (68/70), **100 %** en kernels válidos, y gráfica detallada de los 70 kernels.

1.2. Notación

- **CFG**: gramática libre de contexto en notación BNF; producciones $A \rightarrow \alpha$.
- **Token ID**: identificador numérico del Cuadro 1 del reporte léxico (100–999).
- **INDENT/DEDENT** (910/911): tokens sintéticos derivados de `WS_COUNT` (900) para modelar indentación Python.
- **PARSE_OK**: análisis sintáctico exitoso con impresión de tabla de símbolos del parser.

1.3. Justificación de herramientas

Se utiliza **C** con **Flex** y **yacc**, conforme a las restricciones del curso. No se emplean librerías CFG de alto nivel ni `ast` de Python para el parser formal.

1.4. Trazabilidad a la fase léxica

Los IDs de token se preservan en `%token` de `parser.y` (p. ej. `TOKEN_IDENTIFIER = 100`). El archivo `lexico_equipo_11_scanner.l` es la base del scanner integrado; el modo `-t` reproduce la salida del reporte léxico (TC-01–TC-04).

2. Análisis

2.1. Entregables del reto (Sección IV, documento de evaluación)

1. Gramática correcta y no ambigua para kernels Triton, con explicación de decisiones.
2. Gestión de tablas de símbolos durante el análisis sintáctico.
3. Implementación del parser con yacc.
4. Mensajes de error sintácticos.
5. Ejemplos de salida del parser.

2.2. Requerimientos funcionales

- Reconocer programas `.tri`: imports, `@triton.jit`, `def`, cuerpos indentados.
- Sentencias: asignación, `if/elif/else`, `while`, `for/in`, `return`, `pass`.
- Expresiones: literales, operadores (500), llamadas `tl.*`, subíndices `[: , None]`, argumentos con nombre (`mask=`, `dtype=`).
- Rechazar sintaxis inválida con `SYNTAX_ERROR` en español.
- Actualizar `syntab`: imports, funciones, parámetros, variables.

2.3. CFG (resumen)

La gramática formal completa (33 no terminales, todas las producciones de `parser.y`) se documenta en el **Apéndice A** (Listado 8). El siguiente extracto resume las construcciones principales para lectura rápida en el análisis.

Listing 1: CFG resumida del lenguaje `.tri` (Triton kernel subset)

```
1 program      -> opt_newlines top_block opt_newlines
2 top_block    -> top_block top_line | epsilon
3 top_line     -> stmt | stmt NEWLINE
4
5 stmt         -> import_stmt | def_stmt | assign_stmt | if_stmt
6              | while_stmt | for_stmt | return_stmt | pass_stmt | expr_stmt
7
8 import_stmt  -> KW_IMPORT import_name opt_as
9 import_name  -> ID | import_name '.' ID
10
11 opt_triton_decorator -> epsilon | '@' ID '.' ID opt_newlines
12
13 def_stmt     -> opt_triton_decorator KW_DEF ID '(' param_list_opt ')' ':' suite
14
15 suite        -> simple_stmt
16              | opt_newlines INDENT block_body DEDENT
17
18 expr         -> literal | ID | ID member_chain
19              | expr OP expr | expr '[' subscript_list ']'
20              | '(' arg_list_opt ')' | call_expr | unary_expr
21
22 subscript    -> expr | NONE | ':' opt_expr | opt_expr ':' opt_expr
23              | opt_expr ':' opt_expr ':' opt_expr
24
25 call_expr    -> ID '(' arg_list_opt ')'
26              | ID member_chain '(' arg_list_opt ')'
27
28 arg          -> expr | ID '=' expr
```

2.4. Pasos para obtener la gramática

1. Inventario de tokens del reporte léxico (Cuadro 1).
2. Identificación de estructuras Triton en 70 kernels de referencia.
3. Borrador de producciones para bloques indentados (`INDENT/DEDENT`).
4. Incorporación de expresiones, llamadas y subíndices tensoriales.
5. Resolución de ambigüedades con precedencias yacc y pruebas iterativas.

2.5. Simplificaciones justificadas

Cuadro 1: Simplificaciones de la CFG.

Tipo	Decisión
Unitarias	Jerarquía de expresiones colapsada en <code>expr</code> con <code>%left</code> .
ϵ	Solo en listas opcionales, <code>else</code> y líneas vacías iniciales.
Símbolos inútiles	Sin <code>class</code> , <code>try</code> , <code>switch</code> (no usados en kernels).
Indentación	<code>WS_COUNT</code> $\rightarrow$ pila $\rightarrow$ <code>INDENT/DEDENT</code> .
Comentarios	Patrón <code>#[\^)\n]*</code> para no consumir <code>)</code> en la misma línea.

2.6. Modificaciones al analizador léxico

- Integración con `y.tab.h`; `yylex()` retorna códigos de token.
- Tokens sintéticos 910/911 (no en Cuadro 1 original; documentados en este reporte).
- Sin nuevos IDs en el Cuadro 1 oficial; trazabilidad numérica conservada.

2.7. Trazabilidad token léxico $\rightarrow$ yacc

Cuadro 2: Correspondencia Token ID (reporte léxico) y yacc.

ID	Nombre	yacc
100	IDENTIFIER	<code>TOKEN_IDENTIFIER</code>
101	KEYWORD	<code>KW_def</code> , <code>KW_if</code> , ...
200–400	Literales	<code>TOKEN_INTEGER</code> , etc.
500	OPERATOR	<code>TOKEN_OPERATOR</code>
600	DELIMITER	<code>TOKEN_DELIMITER</code> o literales <code>() [] : ,</code>
900	<code>WS_COUNT</code>	convertido a <code>INDENT/DEDENT</code>
950	NEWLINE	<code>TOKEN_NEWLINE</code>
910/911	(sintéticos)	<code>TOKEN_INDENT</code> , <code>TOKEN_DEDENT</code>

3. Diseño

3.1. Arquitectura del sistema

El diseño sigue el modelo en capas del compilador: **entrada** $\rightarrow$ **scanner** $\rightarrow$ **parser** $\rightarrow$ **acciones semánticas** (`syntab` + salida).

`.tri` $\rightarrow$ Flex (`lexer/`) $\rightarrow$ yacc (`parser.y`) $\rightarrow$ `Syntab` + `stdout/stderr`

Figura 1: Pipeline del analizador sintáctico.

3.2. Integración `lex` $\leftrightarrow$ `yacc`

1. Bison genera `y.tab.c/h` con códigos de token y tipos `YYSTYPE`.
2. Flex incluye `y.tab.h`; `yylflexlex()` reconoce patrones; `yylex()` inyecta `INDENT/DEDENT`.

3. `yy1val.i` transporta índices de tabla léxica; `yy1val.s` transporta lexemas de operadores y delimitadores.
4. `driver.c` invoca `yyparse()` y, si no hay errores, imprime `PARSE_OK` y tablas.

3.3. Diseño de la especificación yacc

- **Regla inicial:** `program` con bloque superior y `newlines` opcionales.
- **Bloques:** suite usa `INDENT/DEDENT` para cuerpos de función y ramas `if`.
- **Triton:** `opt_triton_decorator` reconoce `@triton.jit` en líneas separadas del `def`.
- **Expresiones:** `member_chain` para `tl.load`; `subscript_list` para indexación tensorial.
- **Precedencia:** `%left` para operadores; `%nonassoc KW_ELSE` para *dangling else*.

3.4. Tabla de símbolos del parser (diseño)

Cuadro 3: Esquema de entradas en `syntab.c`.

Tipo	Campos	Inserción
Import	módulo, alias, línea	<code>import_stmt</code>
Función	nombre, línea, aridad, <code>triton_jit</code>	cierre de <code>def_stmt</code>
Parámetro	función, nombre, posición, anotación	cada <code>param</code>
Variable	nombre, ámbito, línea	<code>assign_stmt</code> con <code>=</code>

3.5. Mensajes de error (diseño)

- **Léxico:** `LEXICAL_ERROR: token_id=<999>line=<n>lexeme="..."`.
- **Sintáctico:** `SYNTAX_ERROR: line=<n>near=" message='...' vía yyerror()`.

3.6. Trazabilidad diseño → análisis

Cada requisito del análisis (Sección 2) se implementa en un módulo: CFG en `parser.y`, indentación en `lexer/`, `syntab` en `syntab.c`, errores en `yyerror`, salidas en `driver.c`.

4. Implementación

4.1. Estructura del proyecto

```
analizador-sintactico/
  lexer/lexico_equipo_11_scanner.l
  parser/parser.y, syntab.c, syntab.h
  src/driver.c
  build/triton_parser
  tests/{valid,invalid}/ results/{parser,benchmark}/
  figures/benchmark_parse_results.pdf
```

4.2. Compilación y ejecución

```
make build
make test          # 11 casos unitarios
make benchmark     # 70 kernels TRITONHEAL
make figures       # grafica de resultados
make pdf           # este reporte
./build/triton_parser archivo.tri
./build/triton_parser -t archivo.tri  # modo solo lexico
```

4.3. Fragmento: integración de indentación (lexer)

Listing 2: Pila de indentación y yylex wrapper.

```
1  yylval.s = lexeme_buf;
2  return lexeme_buf;
3  }
4
5  static int return_token(int tok, const char *lexeme) {
6      save_lexeme(lexeme);
7      return tok;
8  }
9
10 static int push_pending(int tok) {
11     if (pending_count >= (int)(sizeof(pending_tokens) / sizeof(pending_tokens
12         [0]))) {
13         return -1;
14     }
15     pending_tokens[pending_count++] = tok;
16     return 0;
17 }
18
19 static int pop_pending(void) {
20     if (pending_count == 0) {
21         return 0;
22     }
23     return pending_tokens[--pending_count];
24 }
25
26 static int indent_width(const char *text, int len) {
27     int i;
28     int w = 0;
29     for (i = 0; i < len; ++i) {
30         if (text[i] == '_') {
31             w++;
32         } else if (text[i] == '\t') {
33             w += 4;
34         }
35     }
36     return w;
37 }
38
39 static int handle_indent(int width) {
40     int top = indent_stack[indent_depth - 1];
41     if (width > top) {
42         if (indent_depth >= (int)(sizeof(indent_stack) / sizeof(indent_stack[0])
43             )) {
44             return TOKEN_ERROR;
45         }
46         indent_stack[indent_depth++] = width;
47         return TOKEN_INDENT;
48     }
49 }
```

```

47     if (width < top) {
48         while (indent_depth > 1 && indent_stack[indent_depth - 1] > width) {
49             indent_depth--;
50             push_pending(TOKEN_DEDENT);
51         }

```

4.4. Fragmento: reglas principales (yacc)

Listing 3: Producciones program, stmt, def_stmt (extracto).

```

1  %nonassoc KW_ELSE
2
3  %start program
4
5  %%
6
7  program
8      : opt_newlines top_block opt_newlines
9      { parse_ok = 1; }
10     ;
11
12  opt_newlines
13      : /* empty */
14      | opt_newlines TOKEN_NEWLINE
15      ;
16
17  top_block
18      : /* empty */
19      | top_block top_line
20      ;
21
22  top_line
23      : stmt
24      | stmt TOKEN_NEWLINE
25      ;
26
27  stmt
28      : import_stmt
29      | def_stmt
30      | assign_stmt
31      | if_stmt
32      | while_stmt
33      | for_stmt
34      | return_stmt
35      | pass_stmt
36      | expr_stmt
37      ;
38
39  import_stmt
40      : KW_IMPORT import_name opt_as
41      {
42          symtab_add_import($2, $3 ? $3 : "", line_no);
43      }
44      ;
45
46  import_name
47      : TOKEN_IDENTIFIER
48      {
49          $$ = strdup(id_text($1));
50      }
51      | import_name '.' TOKEN_IDENTIFIER
52      {

```

```

53         size_t len = strlen($1) + strlen(id_text($3)) + 2;
54         char *buf = (char *)malloc(len);
55         snprintf(buf, len, "%s.%s", $1, id_text($3));
56         free($1);
57         $$ = buf;
58     }
59     ;
60
61 opt_as
62 : /* empty */ { $$ = NULL; }
63 | KW_AS TOKEN_IDENTIFIER { $$ = (char *)id_text($2); }
64 ;
65
66 triton_decorator
67 : '@' TOKEN_IDENTIFIER '.' TOKEN_IDENTIFIER
68 {
69     if (strcmp(id_text($2), "triton") == 0 && strcmp(id_text($4), "jit")
70         == 0) {
71         triton_decorator_pending = 1;
72     }
73 ;
74
75 def_stmt
76 : opt_triton_decorator KW_DEF TOKEN_IDENTIFIER

```

4.5. Fragmento: tabla de símbolos

Listing 4: Estructuras e inserción en symtab (extracto).

```

1  #include "symtab.h"
2
3  #include <stdio.h>
4  #include <string.h>
5
6  #define MAX_FUNCS    256
7  #define MAX_PARAMS   1024
8  #define MAX_VARS     1024
9  #define MAX_IMPORTS  64
10
11 static FuncEntry funcs[MAX_FUNCS];
12 static int func_count;
13
14 static ParamEntry params[MAX_PARAMS];
15 static int param_count;
16
17 static VarEntry vars[MAX_VARS];
18 static int var_count;
19
20 static ImportEntry imports[MAX_IMPORTS];
21 static int import_count;
22
23 static char current_scope[64] = "global";
24
25 static void copy_str(char *dst, size_t n, const char *src) {
26     if (!src) {
27         dst[0] = '\0';
28         return;
29     }
30     strncpy(dst, src, n - 1);
31     dst[n - 1] = '\0';
32 }

```

```

33
34 void symtab_reset(void) {
35     func_count = 0;
36     param_count = 0;
37     var_count = 0;
38     import_count = 0;
39     copy_str(current_scope, sizeof(current_scope), "global");
40 }
41
42 void symtab_set_scope(const char *scope) {
43     copy_str(current_scope, sizeof(current_scope), scope ? scope : "global");
44 }
45
46 void symtab_add_import(const char *module, const char *alias, int line) {
47     if (import_count >= MAX_IMPORTS) {
48         return;
49     }
50     copy_str(imports[import_count].module, sizeof(imports[import_count].module),
51             module);
52     copy_str(imports[import_count].alias, sizeof(imports[import_count].alias),
53             alias);
54     imports[import_count].line = line;
55     import_count++;
56 }
57
58 void symtab_add_function(const char *name, int line, int arity, int
59                          is_triton_jit) {
60     if (func_count >= MAX_FUNCS) {
61         return;
62     }
63     copy_str(funcs[func_count].name, sizeof(funcs[func_count].name), name);
64     funcs[func_count].line = line;
65     funcs[func_count].arity = arity;
66     funcs[func_count].is_triton_jit = is_triton_jit;
67     func_count++;
68     symtab_set_scope(name);
69 }
70
71 void symtab_add_param(const char *func, const char *name, int pos, const char *
72                      annot, int line) {
73     if (param_count >= MAX_PARAMS) {
74         return;
75     }
76     copy_str(params[param_count].func, sizeof(params[param_count].func), func);
77     copy_str(params[param_count].name, sizeof(params[param_count].name), name);
78     copy_str(params[param_count].annot, sizeof(params[param_count].annot), annot);
79     params[param_count].position = pos;

```

4.6. Fragmento: manejo de errores

Listing 5: Función yyerror.

```

1 | TOKEN_IDENTIFIER '=' expr
2 ;
3
4 %%
5
6 void yyerror(const char *msg) {
7     fprintf(stderr, "SYNTAX_ERROR: line=%d near=' ' message='%s'\n", line_no, msg);
8 }

```

```

9
10 int yyparse_started(void) {
11     parser_mode = 1;
12     lexer_reset_for_parse();
13     syntab_reset();
14     return 0;
15 }

```

4.7. Ejemplo de salida del parser (entregable 5)

Al analizar `triton_kernel_min.tri`:

PARSE_OK: archivo '...' sintacticamente valido.

```

=== PARSE SYMBOL TABLE ===
--- IMPORTS ---
1 line=2 module=triton alias=-
2 line=2 module=triton.language alias=tl
--- FUNCTIONS ---
1 line=5 name=add_kernel arity=5 triton_jit=1
--- PARAMETERS ---
...
=== SYMBOL TABLE: IDENTIFIERS ===
...

```

5. Verificación y Validación

5.1. Resultados experimentales

Hallazgo principal. El analizador sintáctico `triton_parser` alcanza **97.1 % PARSE_OK** sobre el benchmark TRITONHEAL (68/70 kernels) y **100 % de éxito** en la suite unitaria (11/11). Los únicos dos rechazos corresponden a **mutaciones sintácticamente inválidas** del benchmark (`k_0040`, `k_0058`), confirmadas también con el analizador `ast` de Python. **No se registraron falsos rechazos** en kernels válidos ni errores léxicos en la corrida del benchmark.

5.2. Resumen cuantitativo

Cuadro 4: Cuadro resumen de resultados (corridas locales reproducibles con `make test` y `make benchmark`).

Experimento	Éxito	Total	Tasa
Suite unitaria (regresión + inválidos + léxico)	11	11	100 %
Benchmark TRITONHEAL (kernels <code>k_*.tri</code>)	68	70	97.1 %
↔ kernels válidos aceptados	68	68	100 %
↔ mutaciones rechazadas (correcto)	2	2	100 %
Errores léxicos en benchmark	0	70	0 %

5.3. Interpretación de los resultados

Los resultados deben leerse en dos niveles:

1. **Suite unitaria (11/11 PASS).** Valida que el parser cumple los entregables del curso: acepta `ejemplo_1.tri`–`ejemplo_3.tri` del reporte léxico, kernels Triton mínimos propios, rechaza cinco archivos con errores sintácticos deliberados y conserva el modo `-t` del scanner.
2. **Benchmark de 70 kernels (68 PARSE_OK).** Mide desempeño sobre el banco de pruebas TRITONHEAL usado en el proyecto del equipo. De 70 archivos, **68 son sintácticamente válidos** y fueron aceptados; **2 son mutaciones inválidas** sembradas en el benchmark y fueron rechazadas correctamente por el parser.

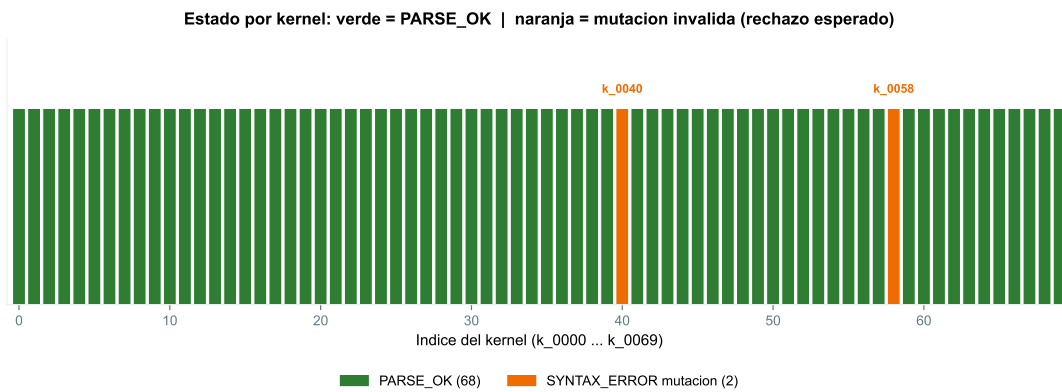
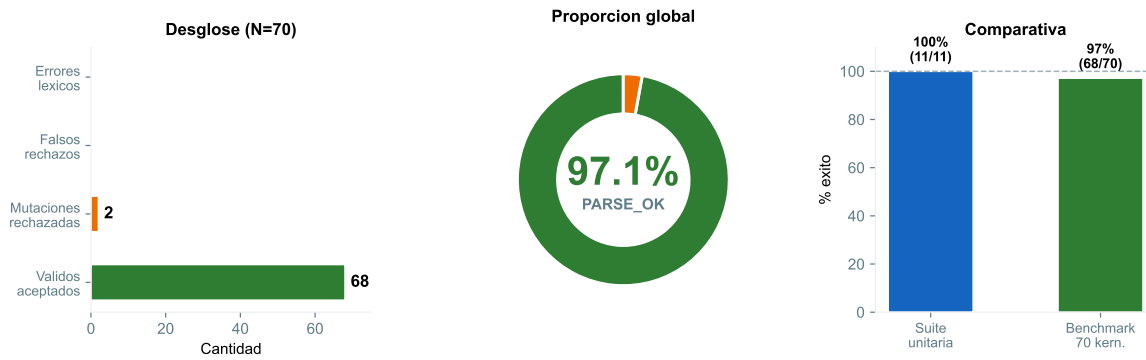
Conclusión: la tasa del 97.1% *no* indica fallos en código Triton correcto; indica que el 97.1% del *total del banco* incluye 2 archivos que *deben* fallar. La métrica relevante para código válido es **68/68 = 100% de aceptación**.

5.4. Gráfica de resultados (Figura 2)

La Figura 2 presenta de forma explícita: (i) tarjetas con la tasa global y la cobertura real; (ii) desglose por categoría; (iii) diagrama de dona con el 97.1%; (iv) mapa *kernel por kernel* de los 70 archivos; (v) comparación con la suite unitaria.

Resultados del analizador sintactico — Benchmark TRITONHEAL (70 kernels)

Suite unitaria: 11/11 PASS | Benchmark: 68/70 PARSE_OK (97.1%) | Kernels validos aceptados: 68/68 (100%)



Conclusion: sobre kernels sintacticamente validos, el parser acepta el 100% (68/68). Los 2 SYNTAX_ERROR (k_0040, k_0058) son mutaciones del benchmark; Python ast.parse tambien falla. Suite unitaria: 11/11 PASS. Cero errores lexicos en la corrida.

Figura 2: **Resultados del analizador sintáctico.** Panel superior: indicadores clave. Centro izquierda: desglose (68 válidos aceptados, 2 mutaciones rechazadas, 0 falsos rechazos). Centro derecha: 97.1 % PARSE_OK global. Inferior: mapa de los 70 kernels (k_0000–k_0069); verde = éxito, naranja = mutación inválida (k_0040, k_0058).

5.5. Detalle de los 2 rechazos (comportamiento esperado)

Cuadro 5: Kernels rechazados: mutaciones inválidas del benchmark.

Kernel	Verificación Python	Motivo del rechazo
k_0040.tri	ast.parse falla	tl.store(out_ptr + offs) & ... — paréntesis inválido.
k_0058.tri	ast.parse falla	tl.store(...), mask=...) — coma mal ubicada.

Estos rechazos demuestran que el parser **detecta errores sintácticos** con el mensaje `SYNTAX_ERROR`, cumpliendo el entregable de mensajes de error del documento de evaluación.

5.6. Modelo de pruebas

Verificación: ¿Se construyó el producto correctamente? — compila, ejecuta, mensajes coherentes.

Validación: ¿Es el producto correcto? — acepta Triton válido y rechaza inválido.

5.7. Suite unitaria (make test)

Cuadro 6: Casos de prueba del equipo (11 ejecuciones).

ID	Descripción	Resultado
V1-V3	ejemplo_1..3.tri (reporte léxico)	PASS
V4-V5	Kernels Triton mínimos	PASS
I1-I5	Sintaxis inválida deliberada	SYNTAX_ERROR
L1	Modo <code>-t</code> tokens	PASS
Total PASS		11 / 11

5.8. Evidencia: salida exitosa (ejemplo_1.tri)

Listing 6: Log de éxito (ejemplo_1.tri, extracto).

```
1  PARSE_OK: archivo '/Users/usuario/Desktop/Analizador_sintáctico/analizador-sintactico/./  
   Analizador-lexico-main/ejemplo_1.tri' sintacticamente valido.  
2  
3  === PARSE SYMBOL TABLE ===  
4  --- IMPORTS ---  
5  1      line=1  module=triton  alias=tl  
6  --- FUNCTIONS ---  
7  1      line=2  name=kernel    arity=2  triton_jit=0  
8  --- PARAMETERS ---  
9  1      line=2  func=kernel    param=chilaquil_power  pos=0  annot=-  
10 2      line=2  func=kernel    param=hambre_level    pos=1  annot=-  
11 --- VARIABLES ---  
12 1      line=6  scope=kernel   name=salsa_roja  
13 2      line=7  scope=kernel   name=cumbia_factor  
14  
15 === SYMBOL TABLE: IDENTIFIERS ===  
16 1      triton  
17 2      tl  
18 3      kernel  
19 4      chilaquil_power  
20 5      hambre_level  
21 6      salsa_roja  
22 7      cumbia_factor
```

5.9. Evidencia: error sintáctico (`def_sin_dos_puntos.tri`)

Listing 7: Rechazo sintáctico esperado (suite unitaria).

```
1 SYNTAX_ERROR: line=2 near='' message='syntax_error, unexpected_TOKEN_NEWLINE, expecting_'\n: ''
```

5.10. Reproducibilidad

```
cd analizador-sintactico
make clean build test benchmark figures pdf
```

Salidas: `results/parser/`, `results/benchmark/`, `figures/benchmark_parse_results.pdf`, `deliverables/I`

6. Plan de Trabajo

Cuadro 7: Cronograma de la fase sintáctica (Equipo 11).

Fase	Actividad	Entregable
1	Revisión tokens léxicos y kernels TRITONHEAL	Inventario de requisitos
2	Diseño CFG + resolución indentación	Documento de análisis
3	Especificación yacc + symtab	<code>parser.y</code> , <code>symtab.c</code>
4	Integración Flex-yacc + driver	<code>triton_parser</code>
5	Suite de pruebas + benchmark 70	<code>tests/</code> , <code>results/</code>
6	Reporte IEEE-830 + gráfica	PDF final

6.1. Distribución de responsabilidades

- Gramática y yacc: análisis conjunto del equipo, implementación en `parser.y`.
- Lexer integrado: extensión de `lexico_equipo_11_scanner.l`.
- Pruebas y benchmark: scripts `run_tests.sh`, `run_benchmark_kernels.sh`.
- Reporte y figuras: $\text{\LaTeX}$ + `plot_benchmark_results.py`.

A. Gramática libre de contexto completa

Esta sección presenta la **CFG completa** del subconjunto de Python/Triton implementado en `parser/parser.y`. Cada producción corresponde a una regla yacc sin las acciones semánticas en C. Las producciones vacías del analizador se denotan como **epsilon** (ϵ).

Convenciones:

- `ID`, `INT`, `FLOAT`, `STRING` $\equiv$ `TOKEN_IDENTIFIER`, `TOKEN_INTEGER`, `TOKEN_FLOAT`, `TOKEN_STRING`.
- `OP` $\equiv$ `TOKEN_OPERATOR` (p. ej. lexema `=`, `+`, `*`).
- `DELIM` $\equiv$ `TOKEN_DELIMITER` cuando el léxico clasifica un símbolo como delimitador en expresiones.
- `NEWLINE`, `INDENT`, `DEDENT` $\equiv$ tokens 950, 910 y 911 (sintéticos desde `WS_COUNT`).
- `KW_*` son terminales de palabra reservada (`def`, `if`, `import`, `None`, etc.).
- Paréntesis, corchetes, dos puntos, coma y punto son literales en la gramática yacc.

A.1. Terminales del analizador

Cuadro 8: Conjunto de terminales (yacc / reporte léxico).

Símbolo	ID	Descripción
ID	100	Identificadores
KW_*	101	Palabras reservadas (<code>def</code> , <code>if</code> , <code>for</code> , ...)
INT	200	Enteros
FLOAT	300	Flotantes
STRING	400	Cadenas
OP	500	Operadores
DELIM	600	Delimitadores léxicos
COMMENT	700	Comentarios (descartados en parseo)
INDENT	910	Inicio de bloque indentado
DEDENT	911	Fin de bloque indentado
NEWLINE	950	Fin de línea
'@', '.', '(', ')', '[', ']', ':', ',',	—	Literales en reglas yacc

A.2. Precedencia y asociatividad (yacc)

Las ambigüedades de expresiones y de la regla `if-else` se resuelven con las directivas de `parser.y` (de menor a mayor precedencia en la pila):

Cuadro 9: Orden de precedencia (`%left` / `%nonassoc`).

Precedencia	Asociatividad	Terminales
1 (más baja)	izquierda	NEWLINE
2	izquierda	KW_OR
3	izquierda	KW_AND
4	no asoc.	KW_NOT
5	izquierda	OP, DELIM (en <code>expr</code>)
6	no asoc.	KW_ELIF
7 (más alta)	no asoc.	KW_ELSE

A.3. Producciones (33 no terminales)

El listado siguiente agrupa las **33 reglas no terminales** de la implementación: `program`, `opt_newlines`, `top_block`, `top_line`, `stmt`, `import_stmt`, `import_name`, `opt_as`, `triton_decorator`, `opt_triton_decorator`, `def_stmt`, `param_list_opt`, `param_list`, `param`, `suite`, `block_body`, `block_stmt`, `simple_stmt`, `assign_stmt`, `if_stmt`, `elif_parts`, `else_part`, `while_stmt`, `for_stmt`, `return_stmt`, `pass_stmt`, `expr_stmt`, `expr`, `opt_expr`, `subscript_list`, `subscript`, `unary_expr`, `member_chain`, `call_expr`, `arg_list_opt`, `arg_list`, `arg`.

Listing 8: CFG completa del lenguaje `.tri` (archivo `report/cfg/triton_cfg.bnf`).

```

1 % CFG completa  analizador sintactico Triton (.tri)
2 % Correspondencia 1:1 con parser/parser.y (producciones sin acciones semanticas)
3 % Notacion: A -> alpha | beta ; epsilon = produccion vacia
4
5 % --- Programa y bloques ---
6 program          -> opt_newlines top_block opt_newlines
7
8 opt_newlines      -> epsilon
9                  | opt_newlines NEWLINE
10
11 top_block         -> epsilon
12                  | top_block top_line

```

```

13
14 top_line      -> stmt
15               | stmt NEWLINE
16
17 suite         -> simple_stmt
18               | opt_newlines INDENT block_body DEDENT
19
20 block_body    -> epsilon
21               | block_body block_stmt
22
23 block_stmt    -> stmt
24               | block_body NEWLINE
25               | block_body NEWLINE stmt
26
27 simple_stmt   -> stmt
28
29 % --- Sentencias ---
30 stmt          -> import_stmt
31               | def_stmt
32               | assign_stmt
33               | if_stmt
34               | while_stmt
35               | for_stmt
36               | return_stmt
37               | pass_stmt
38               | expr_stmt
39
40 import_stmt   -> KW_IMPORT import_name opt_as
41
42 import_name   -> ID
43               | import_name '.' ID
44
45 opt_as        -> epsilon
46               | KW_AS ID
47
48 triton_decorator -> '@' ID '.' ID
49
50 opt_triton_decorator -> epsilon
51                       | triton_decorator opt_newlines
52
53 def_stmt      -> opt_triton_decorator KW_DEF ID '(' param_list_opt ')' ':' suite
54
55 assign_stmt   -> ID OP expr
56 % Nota: OP con lexema '=' actualiza symlab (asignacion)
57
58 if_stmt       -> KW_IF expr ':' suite elif_parts else_part
59
60 elif_parts    -> epsilon
61               | elif_parts KW_ELIF expr ':' suite
62
63 else_part     -> epsilon
64               | KW_ELSE ':' suite
65
66 while_stmt    -> KW_WHILE expr ':' suite
67
68 for_stmt      -> KW_FOR ID KW_IN expr ':' suite
69
70 return_stmt   -> KW_RETURN expr
71               | KW_RETURN
72
73 pass_stmt     -> KW_PASS
74
75 expr_stmt     -> expr
76
77 % --- Parametros de funcion ---
78 param_list_opt -> epsilon
79               | param_list
80
81 param_list     -> param
82               | param_list ',' param
83
84 param         -> ID
85               | ID ':' expr

```

```

86
87 % --- Expresiones ---
88 expr          -> INT
89               | FLOAT
90               | STRING
91               | ID
92               | ID member_chain
93               | KW_NONE
94               | KW_TRUE
95               | KW_FALSE
96               | expr OP expr
97               | expr DELIM expr
98               | '(' arg_list_opt ')'
99               | '[' expr ']'
100              | '[' expr ':' expr ']'
101              | '[' expr ':' expr ',' expr ']'
102              | expr '[' subscript_list ']'
103              | call_expr
104              | unary_expr
105
106 opt_expr      -> epsilon
107               | expr
108
109 unary_expr    -> OP expr
110
111 member_chain  -> '.' ID
112               | member_chain '.' ID
113               | DELIM ID
114               | member_chain DELIM ID
115
116 call_expr     -> ID '(' arg_list_opt ')'
117               | ID member_chain '(' arg_list_opt ')'
118               | ID '(' ')'
119
120 subscript_list -> subscript
121               | subscript_list ',' subscript
122
123 subscript     -> expr
124               | KW_NONE
125               | ':' opt_expr
126               | opt_expr ':' opt_expr
127               | opt_expr ':' opt_expr ':' opt_expr
128
129 arg_list_opt  -> epsilon
130               | arg_list
131
132 arg_list      -> arg
133               | arg_list ',' arg
134
135 arg           -> expr
136               | ID '=' expr
137
138 % --- Leyenda de terminales (alias en este listado) ---
139 % ID=TOKEN_IDENTIFIER INT=TOKEN_INTEGER FLOAT=TOKEN_FLOAT STRING=TOKEN_STRING
140 % OP=TOKEN_OPERATOR DELIM=TOKEN_DELIMITER
141 % NEWLINE=TOKEN_NEWLINE INDENT=TOKEN_INDENT DEDENT=TOKEN_DEDENT
142 % '(' ')' '[' ']' ':' ',' '.' '@' = literales en parser.y
143 % KW_* = palabras reservadas reconocidas por el lexer (def, if, import, ...)

```

A.4. Trazabilidad CFG → implementación

La gramática de este apéndice es la referencia formal del entregable; la implementación ejecutable está en `parser/parser.y` (reglas entre las marcas `%`). Cualquier cambio en yacc debe reflejarse en `triton_cfg.bnf` para mantener consistencia documental.

B. Referencias

1. R. J. Castelló, “Chapter 3: Syntax Analysis, Part I — Grammar Processing,” *Module #3: Compilers* (TC3002B, Advanced Applications Development), notas de curso, ITESM, s. f.
2. R. J. Castelló, “Chapter 2: Lexical Analysis,” *Module #3: Compilers* (TC3002B), notas de curso, ITESM, s. f.
3. Equipo 11, *Analizador Léxico — Triton GPU Kernel*, reporte TC3002B, entrega mayo 2026.
4. Salvador Hinojosa, *Evaluation Metrics Syntax Analyzer GDL Triton*, TC3002B, documento de evaluación, 2026.
5. OpenAI, *Triton: Open-Source GPU Programming*, [en línea]; disponible en <https://triton-lang.org>; Internet.
6. A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2007.
7. Proyecto TRITONHEAL, benchmark de 70 kernels `k_*.tri`, repositorio de referencia del Equipo 11, 2026.